commands

```
mov rax, 60         ; move the value 60 into the rax register
mov rdi, 0          ; move the value 0 into the rdi register
                    ; (for example as the exit code)
syscall             ; make the syscall
add rax, rdi        ; add the value in rdi to the value in rax
sub rax, 1          ; subtract 1 from the value in rax
mul rax, 2          ; multiply the value in rax by 2
div rax, 3          ; divide the value in rax by 3
cmp rax, rdi        ; compare the value in rax with the value in rdi
                    ; (zero flag is set if they are equal)
test rax, rdi       ; perform a bitwise AND between rax and rdi
                    ; (zero flag is set if the result is zero)
movsx rax, rdi      ; move the value in rdi into rax with sign
                    ; extension (top bits of rax are filled with the
                    ; sign bit of rdi)
inc rax             ; increment the value in rax by 1
dec rax             ; decrement the value in rax by 1
neg rax             ; negate the value in rax (two's complement)
not rax             ; perform a bitwise NOT on the value in rax
and rax, rdi        ; perform a bitwise AND between rax and rdi
or rax, rdi         ; perform a bitwise OR between rax and rdi
xor rax, rdi        ; perform a bitwise XOR between rax and rdi
shl rax, 1          ; shift the value in rax left by 1 bit
shr rax, 1          ; shift the value in rax right by 1 bit
ror rax, 1          ; rotate the value in rax right by 1 bit
rol rax, 1          ; rotate the value in rax left by 1 bit
```

build

for normal build, you can use the following commands:

```
as -o main.o main.s
ld -o main main.o
```

to build a library, you can use the following commands:

```
as -o lib.o lib.s
ld -o lib.so lib.o -shared
```